

**SIMATS ENGINEERING**  
**CO4 ASSESSMENT TOOL 3**

**CASE STUDY**

COURSE CODE: ITA1404

COURSE NAME: Ethical Hacking

**COURSE OUTCOME COVERED**

CO4: Implement network security testing methods by performing web server attacks, analyzing web application vulnerabilities, and assessing wireless network security using Kali Linux. (BT4, BT5)

Assessment Tool Weightage: 35%

QUESTIONS: 5

Total Marks:100

Code of Conduct:

**I S Venkata Abhishek(192311053)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:Abhishek

**COMPREHENSIVE SECURITY ASSESSMENT REPORT**

**Introduction**

Modern organizations depend heavily on web applications, wireless networks, and Internet-facing servers. Vulnerabilities in any of these components can allow attackers to obtain unauthorized access, steal confidential information, manipulate transactions, or disrupt services. A proper security assessment therefore requires examination of the complete attack surface, including network infrastructure, web servers, applications, authentication mechanisms, wireless configuration, and user behavior.

Kali Linux provides several security-testing tools that can be used in an authorized laboratory or penetration-testing environment. Common tools include Nmap for network discovery and service enumeration, Burp Suite for web-application testing, Nikto for web-server assessment, SQLmap for controlled SQL-injection validation, Wireshark for packet analysis, and Aircrack-ng for authorized wireless security assessment.

The following case studies analyze the possible attack paths, assessment methodology, security impact, and recommended defense mechanisms.

## **CASE STUDY 1: E-Commerce Web Application on Apache/Linux**

### **1.1 Scenario Analysis**

The e-commerce company operates a web application on an Apache server running Linux. Customers have reported unauthorized transactions and abnormal account activity. Investigation has identified suspicious HTTP requests, attempts to access hidden directories, malicious scripts inserted into input fields, and an outdated server version.

The major attack vectors are:

1. Outdated Apache/server components
2. Directory and file enumeration
3. Cross-Site Scripting (XSS)
4. Injection through application inputs
5. Weak input validation
6. Inadequate authentication/session security
7. Missing Web Application Firewall
8. Possible insecure server configuration
9. Insufficient logging and monitoring
10. Potential exploitation of vulnerable third-party components

The combination of these weaknesses creates a high-risk environment because an attacker may first discover the application structure and then exploit an application or server vulnerability.

### **1.2 Reconnaissance Phase**

The first phase should be reconnaissance. Testing must be performed only against the organization's authorized test server.

## **Nmap**

Nmap can be used to identify:

- Open TCP ports
- Running services
- Service versions
- Operating-system information where permitted
- Potentially exposed administrative services

For example, in an authorized lab, an analyst can perform a basic service-discovery scan against the assigned target.

The objective is to determine whether services such as HTTP/HTTPS, SSH, database services, or unnecessary administrative interfaces are exposed.

### **What the Analyst Looks For**

| <b>Finding</b>                         | <b>Security Significance</b>    |
|----------------------------------------|---------------------------------|
| <b>HTTP available</b>                  | Web application exposed         |
| <b>HTTPS unavailable/misconfigured</b> | Possible data exposure          |
| <b>Outdated service version</b>        | Potential known vulnerabilities |
| <b>Unnecessary open ports</b>          | Increased attack surface        |
| <b>Exposed database service</b>        | High-risk configuration         |

## **1.3 Web Server Assessment**

### **Nikto**

Nikto can be used in an authorized environment to identify common web-server configuration problems.

The assessment can identify:

- Outdated server components
- Dangerous HTTP methods
- Default files
- Misconfigured directories
- Missing security headers
- Potentially exposed files

- Known web-server configuration issues

Nikto findings should not automatically be treated as confirmed vulnerabilities. Each finding must be manually validated.

## 1.4 Directory and Resource Enumeration

Suspicious requests attempting to access hidden directories indicate possible directory enumeration.

The analyst can use an authorized content-discovery tool such as **Gobuster** or **ffuf** to determine whether unintended resources are publicly accessible.

The assessment should specifically check for:

- /admin
- Backup files
- Configuration files
- Development directories
- Temporary files
- Source-code remnants
- Log files
- Test pages

Sensitive directories should not be publicly accessible.

## 1.5 Web Application Testing Using Burp Suite

Burp Suite can be used as an interception proxy between the browser and the authorized test application.

The analyst should:

1. Configure the browser to use Burp Suite.
2. Browse the application normally.
3. Capture requests and responses.
4. Identify parameters and input fields.
5. Examine authentication and session-management mechanisms.
6. Test input fields using harmless test payloads.
7. Check whether application responses correctly encode user-controlled data.
8. Examine security headers and cookie attributes.

## 9. Document confirmed findings.

Potential testing areas include:

- Login forms
- Search fields
- Product-review fields
- Address fields
- Checkout parameters
- Profile information
- URL parameters

### 1.6 XSS Assessment

The malicious scripts found in input fields suggest a possible Cross-Site Scripting vulnerability.

Three common categories are:

#### **Reflected XSS**

Malicious input is reflected immediately in an HTTP response.

#### **Stored XSS**

Malicious input is stored by the application and later displayed to other users.

#### **DOM-Based XSS**

Client-side JavaScript processes attacker-controlled data in an unsafe manner.

For a shopping application, stored XSS is particularly serious because malicious content entered into product reviews, profiles, or other persistent fields may affect multiple customers.

### 1.7 Outdated Software Assessment

The outdated Apache version should be identified and compared against trusted vulnerability databases and vendor security advisories.

The analyst should determine:

- Exact installed version
- Operating-system version
- Installed modules
- Third-party libraries

- Application framework version
- Known security advisories
- Available security updates

Outdated software should not be exploited against the production environment merely to prove a vulnerability. Version identification and controlled validation in a test environment are normally sufficient.

## **1.8 Exploitation Validation**

Exploitation should only be performed against an isolated and explicitly authorized test copy.

The objective is to demonstrate the security impact without:

- Modifying production data
- Stealing real customer information
- Performing destructive actions
- Creating persistent malware
- Disrupting transactions

A safe proof-of-concept can demonstrate that a vulnerable parameter accepts unexpected input or that a known vulnerable component is present.

## **1.9 Layered Defense Strategy**

### **Server Hardening**

- Update Apache and Linux regularly.
- Remove unnecessary modules.
- Disable directory listing.
- Remove default/test files.
- Use secure file permissions.
- Disable unnecessary HTTP methods.
- Run services with least privilege.
- Protect configuration files.
- Enable HTTPS.
- Use strong TLS configuration.

### **Secure Coding**

- Validate all input.
- Use allowlists wherever possible.
- Encode output according to its context.
- Use parameterized database queries.
- Implement secure session management.
- Avoid displaying sensitive errors.
- Validate uploaded files.
- Apply secure coding standards.

## **WAF**

A Web Application Firewall can detect and block common malicious HTTP requests.

## **Network Security**

- Use firewalls.
- Separate web, application, and database servers.
- Restrict database access.
- Use IDS/IPS.
- Implement network segmentation.
- Monitor abnormal traffic.

## **Monitoring**

Centralized logging should record:

- Authentication failures
- Administrative actions
- Suspicious requests
- Error rates
- Unexpected HTTP methods
- Repeated directory enumeration
- Abnormal transaction behavior

## **1.10 Conclusion**

The incident is likely the result of multiple weaknesses rather than a single vulnerability. Outdated software, insufficient input validation, insecure application design, and lack of WAF protection create opportunities for attackers. A defense-in-depth strategy combining

server hardening, secure coding, WAF protection, network segmentation, monitoring, and regular vulnerability assessments can significantly reduce the risk.

## **CASE STUDY 2: Financial Web Application – SQL Injection and XSS**

### **2.1 Scenario Analysis**

The financial application allows users to authenticate, view account information, and perform transactions. The application has SQL injection and XSS vulnerabilities. Error messages expose database information and user inputs are not correctly sanitized.

This creates a serious security risk because financial applications process highly sensitive information and perform high-value transactions.

### **2.2 SQL Injection**

SQL injection occurs when untrusted user input is incorporated into database queries without proper parameterization.

A vulnerable application may unintentionally allow an attacker to influence database operations.

#### **Possible Impact**

An attacker could potentially:

- Read unauthorized database records
- Modify information
- Bypass application logic
- Access account information
- Damage database contents
- Potentially affect application availability

The severity depends on database permissions and the application's architecture.

### **2.3 XSS**

XSS occurs when untrusted data is included in web pages without appropriate output encoding.

An attacker could potentially cause malicious client-side content to execute in another user's browser.

### **Possible Impact**

- Session compromise
- Phishing
- Unauthorized actions
- Data theft from the application context
- Account manipulation
- Loss of user trust

Stored XSS is especially dangerous when malicious content is displayed to many users.

## **2.4 Security Audit Methodology**

### **Phase 1 – Reconnaissance**

The analyst identifies:

- Application technologies
- Web-server information
- Entry points
- Login mechanisms
- URL parameters
- Forms
- API endpoints
- Cookies
- Session mechanisms

Nmap can be used for authorized service enumeration.

### **Phase 2 – Burp Suite Assessment**

Burp Suite can be used to intercept application traffic.

The analyst should:

1. Log into the test application.
2. Browse all available functions.
3. Capture requests.
4. Identify user-controlled parameters.

5. Send requests to Burp's testing components.
6. Compare application responses.
7. Test input validation.
8. Examine session cookies.
9. Document confirmed vulnerabilities.

Burp Repeater is useful for manually modifying a request and observing how the application responds.

### **Phase 3 – SQL Injection Validation**

SQLmap can be used in a controlled environment to validate suspected SQL injection.

The safe methodology is:

1. Identify an authorized test request.
2. Capture the request using Burp Suite.
3. Determine the potentially vulnerable parameter.
4. Provide the request to SQLmap.
5. Allow SQLmap to test whether the parameter is injectable.
6. Confirm the finding.
7. Avoid destructive database operations.
8. Avoid extracting real customer data.
9. Document the vulnerable parameter and evidence.

SQLmap should be used only against systems where explicit authorization has been provided.

### **Phase 4 – XSS Validation**

Burp Suite can be used to test whether application input is safely handled.

The analyst should check:

- Reflected parameters
- Stored input
- Search fields
- Profile fields
- Transaction descriptions
- Error pages

- API responses
- Client-side JavaScript

A harmless proof-of-concept should be used to confirm whether user-controlled content is interpreted as executable markup/script rather than safely encoded text.

## 2.5 CIA Triad Analysis

| Security Property       | SQL Injection                                  | XSS                                         |
|-------------------------|------------------------------------------------|---------------------------------------------|
| <b>Confidentiality</b>  | Database/customer information may be exposed   | Session or application data may be exposed  |
| <b>Integrity</b>        | Database records may potentially be modified   | Web content/user actions may be manipulated |
| <b>Availability</b>     | Database/application services may be disrupted | Usually lower direct availability impact    |
| <b>Overall Severity</b> | Critical/High                                  | High                                        |

## 2.6 Mitigation Strategies

### Input Validation

Use allowlists and strict validation for expected input.

### Parameterized Queries

Database operations should use prepared statements rather than dynamically constructing SQL queries.

### Output Encoding

Encode untrusted data according to its output context.

### Content Security Policy

A strong CSP can reduce the impact of some XSS attacks.

### Secure Session Handling

Use:

- Secure cookies
- HttpOnly cookies
- SameSite cookie controls
- Appropriate session expiration

- Session rotation after authentication
- Server-side session invalidation

## **Error Handling**

Do not expose:

- SQL queries
- Database names
- Table structures
- Stack traces
- File paths
- Internal configuration

Instead, provide generic user-facing errors and record detailed information securely in server logs.

## **Additional Controls**

- WAF
- MFA
- IDS/IPS
- SIEM
- Secure code review
- SAST/DAST
- Regular penetration testing
- Database least privilege

## **2.7 Conclusion**

SQL injection and XSS can seriously compromise financial applications. SQL injection primarily threatens database confidentiality and integrity, while XSS can compromise users, sessions, and application integrity. Secure coding, parameterized queries, output encoding, secure sessions, proper error handling, WAF protection, and continuous security testing are essential.

## **CASE STUDY 3: Corporate Wi-Fi Evil Twin Attack**

### **3.1 Scenario Analysis**

The corporate wireless network uses WPA2-PSK. Employees report unusual login prompts and slow network performance. Investigation identifies a rogue access point using the same SSID as the official network.

This is commonly known as an **Evil Twin** attack.

### **3.2 How the Evil Twin Attack Works**

The attack generally follows this sequence:

1. The attacker identifies the organization's wireless SSID.
2. A fraudulent access point is configured to imitate the legitimate SSID.
3. The fraudulent AP is positioned where employees can receive a strong signal.
4. Some devices connect automatically or users manually select the stronger/familiar network.
5. The attacker presents a fraudulent authentication or portal page.
6. Users may enter credentials.
7. If traffic is not properly protected, the attacker may observe or manipulate traffic.
8. Stolen credentials can potentially be used for further attacks.

The attack is especially effective when users trust the SSID name instead of authenticating the wireless infrastructure itself.

### **3.3 Weaknesses in WPA2-PSK**

The use of WPA2-PSK creates several weaknesses in a corporate environment.

#### **Shared Credential**

Employees use the same wireless secret.

#### **Poor Accountability**

It becomes difficult to determine which employee/device used the shared key.

#### **Credential Distribution**

If the PSK is exposed, changing it across many devices can be difficult.

#### **No Strong Individual Authentication**

A corporate network should preferably authenticate individual users and/or devices.

### **3.4 Aircrack-ng Assessment**

Aircrack-ng can be used in an authorized wireless laboratory to:

- Discover nearby wireless networks
- Examine wireless security configuration
- Analyze wireless captures
- Assess authentication mechanisms
- Identify suspicious duplicate SSIDs

The tool should be used only on authorized networks.

The analyst should compare:

- BSSID
- Channel
- Encryption type
- Signal strength
- Vendor information
- Authentication configuration

A duplicate SSID associated with an unexpected BSSID can indicate a rogue AP.

### **3.5 Wireshark Analysis**

Wireshark can be used to analyze authorized packet captures.

The analyst can examine:

- Beacon frames
- Probe requests/responses
- Authentication traffic
- Association frames
- DNS traffic
- HTTP traffic
- TLS connections
- Repeated disconnections
- Unexpected wireless behavior

Indicators of an Evil Twin may include:

- Same SSID with different BSSID
- Unexpected vendor/device

- Different security configuration
- Suspicious authentication behavior
- Abnormal signal strength
- Unexpected gateway/DNS configuration

### **3.6 Countermeasures**

#### **WPA3-Enterprise**

The organization should migrate toward WPA3-Enterprise where supported.

#### **802.1X**

Use individual authentication rather than a shared PSK.

#### **Certificate Validation**

Managed devices should validate the authentication server's certificate correctly.

#### **Wireless IDS/IPS**

A WIDS/WIPS can identify:

- Rogue APs
- Duplicate SSIDs
- Suspicious wireless behavior
- Unauthorized wireless devices

#### **Network Segmentation**

Separate:

- Employee devices
- Guest devices
- IoT
- Administrative systems
- Sensitive servers

#### **User Awareness**

Employees should be trained to:

- Verify corporate Wi-Fi connections.
- Report unexpected login pages.
- Never ignore certificate warnings.

- Avoid entering corporate credentials into suspicious portals.
- Contact IT when wireless behavior changes unexpectedly.

### 3.7 Conclusion

The primary weakness was reliance on WPA2-PSK and insufficient rogue-access-point detection. Enterprise authentication, certificate validation, WIDS/WIPS, segmentation, and user awareness provide significantly stronger protection against Evil Twin attacks.

## CASE STUDY 4: Wireless-to-Web Application Attack Chain

### 4.1 Scenario Analysis

The organization provides a web-based employee management system over Wi-Fi. The wireless network uses weak passwords and the web application has poor authentication and session management.

This creates a chained attack:

**Weak Wi-Fi → Network Access → Reconnaissance → Web Discovery → Web Vulnerability → Privilege Escalation → Confidential Data Access**

### 4.2 Phase 1 – Wireless Assessment

The assessment begins by determining:

- Wireless encryption
- Authentication method
- Password policy
- Network segmentation
- Connected devices
- Access-point configuration
- Guest/employee separation

Aircrack-ng can be used in an authorized lab to assess wireless security configuration.

The main finding is the use of weak shared authentication.

### 4.3 Phase 2 – Network Reconnaissance

Once authorized network access is established, Nmap can identify systems and services in the test environment.

The analyst looks for:

- Web servers
- SSH
- Database services
- Management interfaces
- Unexpected services
- Service versions

The objective is to understand the attack surface, not to attack unrelated systems.

#### **4.4 Phase 3 – Web Application Enumeration**

Burp Suite can be used to examine the employee-management application.

The analyst identifies:

- Login pages
- User roles
- Session cookies
- Employee records
- Administrative functions
- API endpoints
- Input parameters

The analyst should determine whether ordinary users can access functions intended for administrators.

#### **4.5 Phase 4 – Vulnerability Assessment**

The application should be checked for:

- Broken authentication
- Broken access control
- Session fixation
- Weak session tokens
- IDOR
- SQL injection
- XSS

- CSRF
- Insecure API endpoints
- Privilege escalation

Automated scanners can help identify potential vulnerabilities, but manual verification is necessary.

#### 4.6 Phase 5 – Privilege Escalation Assessment

Privilege escalation occurs when a lower-privileged user obtains functionality or information intended for a higher-privileged user.

The assessment should verify:

- Server-side authorization
- Role-based access control
- Object-level authorization
- Administrative endpoints
- API authorization
- Session-role handling

For example, changing a user identifier in a request should not allow access to another employee's information.

The tester should validate authorization using test accounts with different roles rather than real employee accounts.

#### 4.7 Kali Linux Tools

| Phase                  | Tool          | Purpose                        |
|------------------------|---------------|--------------------------------|
| Wireless assessment    | Aircrack-ng   | Authorized wireless analysis   |
| Network discovery      | Nmap          | Host/service enumeration       |
| Web proxy/testing      | Burp Suite    | Request and response analysis  |
| Web-server assessment  | Nikto         | Server configuration checks    |
| Directory discovery    | Gobuster/ffuf | Authorized content discovery   |
| Packet analysis        | Wireshark     | Network traffic analysis       |
| Vulnerability analysis | SearchSploit  | Research known vulnerabilities |

#### 4.8 Defense-in-Depth Framework

## **Wireless Layer**

- WPA3-Enterprise
- 802.1X
- Strong authentication
- WIDS/WIPS
- Rogue AP detection
- Network segmentation

## **Network Layer**

- VLANs
- Firewalls
- NAC
- IDS/IPS
- Least-privilege communication
- Secure management network

## **Application Layer**

- Strong authentication
- MFA
- RBAC
- Secure session management
- Input validation
- Output encoding
- Parameterized SQL queries
- CSRF protection

## **Server Layer**

- Regular patching
- Secure configuration
- Minimal services
- File-permission controls
- Host firewall
- Endpoint monitoring

## Monitoring Layer

- Centralized logs
- SIEM
- Authentication monitoring
- Network anomaly detection
- Incident-response procedures

## 4.9 Conclusion

The incident demonstrates that security weaknesses can be chained together. A weak wireless password provides initial network access, while weak web authentication and authorization allow an attacker to move toward sensitive employee information. Defense-in-depth is therefore essential because failure of one control should not result in complete compromise.

## CASE STUDY 5: Wi-Fi Deauthentication Attacks

### 5.1 Scenario Analysis

The university experiences frequent Wi-Fi disconnections during peak hours. Investigation shows repeated deauthentication activity targeting access points.

Deauthentication attacks exploit weaknesses in traditional IEEE 802.11 management-frame protection. An attacker can transmit forged management frames that cause legitimate wireless clients to disconnect from an access point.

The result is primarily an **availability attack**.

### 5.2 How Deauthentication Attacks Work

A simplified attack sequence is:

**Legitimate Client ↔ Access Point**

↓

**Attacker observes wireless environment**

↓

**Attacker generates forged management traffic**

↓

**Client receives apparent deauthentication information**

↓

**Client disconnects**

↓

**User reconnects**

↓

**Attack repeats**

Continuous repetition can result in a denial-of-service condition.

### **5.3 Why the Attack Works**

Traditional Wi-Fi management frames historically lacked sufficient cryptographic protection.

An attacker does not necessarily need to break the encryption protecting application data. Instead, the attacker targets the wireless management process itself.

This makes the attack primarily a weakness in wireless availability and management-frame protection.

### **5.4 Kali Linux Analysis**

Testing should be performed only in an isolated university security laboratory or another explicitly authorized environment.

#### **Aircrack-ng**

Aircrack-ng tools can be used to observe wireless networks and analyze wireless security configurations.

The analyst can examine:

- Access points
- Clients
- Channels
- Encryption
- Authentication
- Wireless activity

For controlled simulation, a dedicated test AP and test client should be used.

#### **Wireshark**

Wireshark can analyze an authorized packet capture and identify repeated deauthentication frames.

The analyst can look for:

- Repeated deauthentication frames
- Unexpected source addresses
- High frequency of management frames
- Sudden client disconnections
- Association/reassociation patterns

## **WIDS/WIPS**

A wireless intrusion detection system can identify abnormal management-frame activity and generate alerts.

## **5.5 Impact on CIA Triad**

### **Confidentiality**

Normally, a deauthentication attack does not directly disclose encrypted application data.

**Impact: Low to Medium**

### **Integrity**

The attack primarily affects connectivity rather than modifying application data.

**Impact: Low**

### **Availability**

This is the major impact.

Users may be unable to:

- Access websites
- Use cloud services
- Attend online classes
- Access university applications
- Submit assignments
- Communicate with internal systems

**Impact: Critical**

## **5.6 Countermeasures**

### **Protected Management Frames**

Enable **802.11w/Protected Management Frames** where supported.

### **WPA3**

WPA3 provides stronger wireless security and requires appropriate management-frame protection behavior.

### **WIDS/WIPS**

Deploy wireless intrusion detection and prevention systems to detect abnormal management-frame activity.

### **Wireless Monitoring**

Monitor:

- Deauthentication rates
- Rogue APs
- Channel interference
- Unexpected devices
- Abnormal management traffic

### **Network Design**

Use multiple access points and appropriate channel planning so that a localized wireless problem does not affect the entire campus.

### **Access Point Configuration**

Keep access points updated with current firmware and security patches.

### **Incident Response**

When an attack is detected:

1. Identify affected APs.
2. Determine whether the activity is malicious or caused by configuration problems.
3. Preserve relevant logs and packet captures.
4. Locate unauthorized wireless equipment where possible.
5. Apply wireless security controls.
6. Notify the responsible security team.
7. Continue monitoring.

## **5.7 Risk Assessment**

| <b>Factor</b> | <b>Assessment</b> |
|---------------|-------------------|
|---------------|-------------------|

|                               |                            |
|-------------------------------|----------------------------|
| <b>Attack Type</b>            | Wireless denial of service |
| <b>Primary Target</b>         | Availability               |
| <b>Confidentiality Impact</b> | Low                        |
| <b>Integrity Impact</b>       | Low                        |
| <b>Availability Impact</b>    | Critical                   |
| <b>Business Impact</b>        | High                       |
| <b>Recommended Priority</b>   | Immediate                  |

## 5.8 Conclusion

Deauthentication attacks exploit weaknesses in wireless management-frame security and can repeatedly disconnect legitimate users. The university should implement Protected Management Frames, WPA3, WIDS/WIPS, updated access points, wireless monitoring, proper RF planning, and an effective incident-response process.

## OVERALL SECURITY RECOMMENDATIONS

The five scenarios demonstrate that web, wireless, and network security should be treated as interconnected security layers.

### 1. Wireless Security

Organizations should:

- Prefer WPA3-Enterprise.
- Use WPA2-Enterprise where WPA3 is unavailable.
- Implement 802.1X.
- Avoid shared PSKs for enterprise networks.
- Enable Protected Management Frames.
- Deploy WIDS/WIPS.
- Detect rogue access points.
- Segment wireless networks.
- Regularly update AP firmware.

### 2. Web Application Security

Applications should implement:

- Strong authentication
- MFA

- Secure session management
- Server-side authorization
- Input validation
- Output encoding
- Parameterized queries
- CSRF protection
- Secure cookies
- Content Security Policy
- Security headers
- Generic error messages

### **3. Server Security**

Servers should be:

- Regularly patched
- Hardened
- Monitored
- Protected by host firewalls
- Configured with least privilege
- Free from unnecessary services
- Regularly vulnerability-tested

### **4. Network Security**

Organizations should implement:

- Firewalls
- VLAN segmentation
- NAC
- IDS/IPS
- SIEM
- Network monitoring
- Secure management networks
- Restricted administrative access

### **5. Security Monitoring**

Centralized monitoring should correlate:

- Wireless events
- Authentication events
- Web-server logs
- Application logs
- Firewall events
- Endpoint alerts
- Administrative activity

This allows security teams to detect attack chains instead of investigating individual events in isolation.

## **PENETRATION TESTING METHODOLOGY**

A comprehensive assessment should follow the following lifecycle:

### **1. Planning and Authorization**

↓

### **2. Reconnaissance**

↓

### **3. Scanning and Enumeration**

↓

### **4. Vulnerability Identification**

↓

### **5. Controlled Validation**

↓

### **6. Risk and Impact Analysis**

↓

### **7. Remediation**

↓

### **8. Retesting**

↓

### **9. Final Security Report**

All testing must remain within the approved scope. Production systems, real customer information, unrelated networks, and third-party infrastructure must not be targeted.

## CONCLUSION

The five case studies demonstrate that modern cybersecurity incidents frequently involve multiple weaknesses rather than one isolated vulnerability. In the e-commerce scenario, outdated Apache software, insecure input handling, exposed directories, and the absence of a WAF create multiple attack opportunities. In the financial application, SQL injection and XSS can threaten sensitive financial information, user accounts, and transaction integrity. The Evil Twin scenario demonstrates how weak wireless authentication can allow attackers to impersonate legitimate networks and target user credentials.

The wireless-to-web attack chain demonstrates that compromising one security layer can provide a starting point for attacks against another layer. Finally, the university deauthentication scenario demonstrates that wireless management vulnerabilities can severely affect availability even when application data remains encrypted.

A strong cybersecurity program should therefore use **defense in depth**. WPA3-Enterprise, 802.1X, Protected Management Frames, WIDS/WIPS, network segmentation, secure coding, parameterized database queries, output encoding, MFA, secure session management, patch management, WAF, IDS/IPS, SIEM, endpoint security, and continuous vulnerability assessment should be implemented together.

Kali Linux is valuable for authorized security assessments because it provides tools such as **Nmap, Burp Suite, Nikto, SQLmap, Aircrack-ng, Wireshark, Gobuster, ffuf, and SearchSploit**. However, these tools should be used responsibly and only against systems for which explicit testing authorization has been obtained.

The ultimate objective of penetration testing is not simply to demonstrate that a system can be attacked, but to **identify weaknesses, measure their risk, recommend effective controls, and verify that the vulnerabilities have been successfully remediated**.

